European Union Cyber Resilience Act (CRA)

Project Phase #2

Due September 27

Q1 Common Problem Selection

Shared work - complete Sections 1-6 with your pod. Phase 2 establishes a shared specification that each team can independently implement and test. Each team submits its own report: use the same agreed content for Sections 1-6, then complete Sections 7-9 independently. Include sources in Section 10. Reuse and refine Phase 1 findings where relevant.

Q1.1 Selected problem

***(3-4 sentences)**: Identify the users, the task they need to accomplish, and the limitation in the current approaches. Explain how software could improve the outcome and connect the problem to specific regulatory obligations. Cite the relevant provisions.*

Manufacturers subject to the CRA must report every actively exploited vulnerability through three reports. An early warning within 24 hours of becoming aware of the issue, in 72 hours the manufacturer must share a vulnerability notification, and within 14 days of the fix being available they must share a final report [1, Art. 14(1)-(2)]. Reports are submitted by individuals (researchers, customers, internal staff, etc.) to manufactures via the contact point they must provide [1, Art. 13(17); Annex I, Part II(6)]. Our system would track these reports separate of the internal ticketing system (where deadlines are tracked informally and can be missed, especially if a report arrives off work hours) to ensure that every requirement (triage decision, awareness time, deadlines, etc.) remains visible and maintains a trail of compliance.

Q1.2 Selection rationale

***(3-5 sentences)**: Name the Phase 1 alternatives considered and explain why the pod selected or combined these ideas. Explain why the problem involves meaningful engineering work, fits the semester, and permits independent implementations whose outputs can be compared.*

In Phase 1, Team 17 focused on a CRA Reporting Manager for Article 14 deadlines and a Release Component Index for mapping SBOMs to shipped releases, while Team 21 focused on how vulnerability reports would reach the manufacture to even kick off that process. We dropped SBOM matching because both teams found that unreliable component identifiers would be an issue, and we also dropped integrating with an existing ticketing product because cross-team tests would rely on outside accounts and services. Combining both teams' focus led to the creation of a system that gives one lifecycle for a reported issue that is tracked from intake to final report. The system will emphasize tracking state, time handling, validation, and audit integrity (all engineering issues) to help work fit in the semester. Outputs of the system are deadlines, statuses, missing fields, error codes, and drafts of each required report, which can be checked against hand computed answers because test inputs can be shared across teams.

Q1.3 Pod agreement:

Record the meeting date, participating teams and representatives, and the agreed specification version or date. State any remaining disagreement that affects the common scope or testing behavior and how the pod will resolve it. If no disagreement remains, state that.

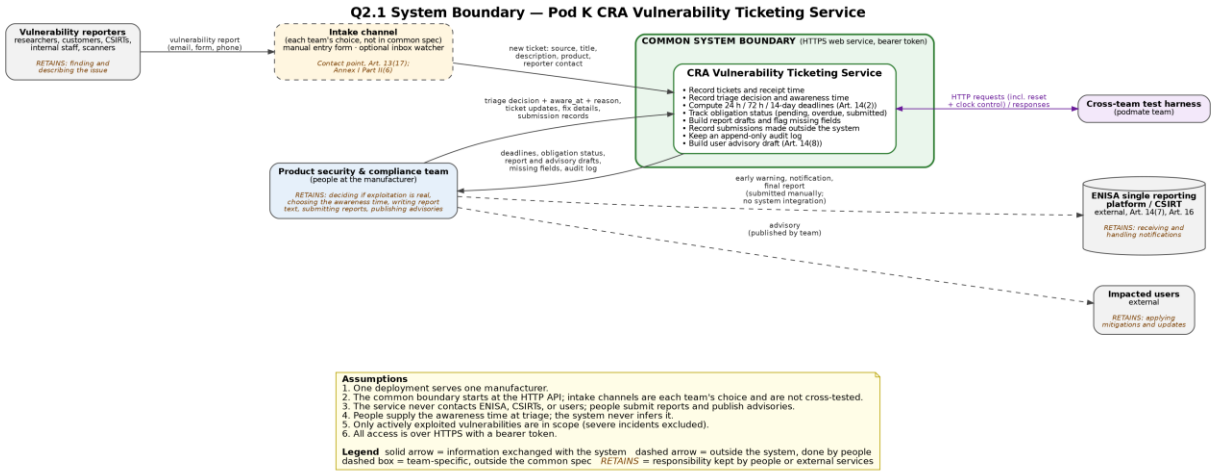
Team 17 (Quincy DeKlerk, Richard Lin, Ann Zuo) and Team 21(Aashik Suresh, Andrew Plotnikov, Ryan Becker, Conrad Wong) met asynchronously over the week of September 20, 2026. Pod spec was agreed upon as of September 25, 2026. No disagreement remains open.

Q2 Common Scope and Cost Estimate

Shared work - complete with your pod.

Q2.1 System boundary

Insert a context or boundary diagram showing the common system, users, external systems, and the main information exchanged across the boundary. Label responsibilities retained by people or external services. Use any clear notation; identify assumptions the diagram relies on.



Q2.2 Included and excluded scope

***(3-5 bullets each)**: State the capabilities every team will implement and the capabilities explicitly excluded. Identify limits on supported data, users, formats, or operating conditions where they constrain the project. Reference the regulatory obligations the included capabilities support.*

Included

- Ticket intake through common HTTPS interface. Receipt time set by the receiving system [1, Art. 13(17); Annex I, Part II(6)].

- Each ticket will be assessed and marked as reportable (actively exploited – awareness time and reason are supplied by an individual) or closed as not reportable [1, Art. 3(42); Art. 14(1)].
- Deadline computation and live status for the early warning, vulnerability notification, and final report [1, Art. 14(2)(a)-(c)].
- Report drafts built from ticket fields with checks for missing required content, recording of reports submitted outside the system, and a per-ticket audit log that is append-only [1, Art. 14(2); Annex I, Part II(1)].
- A draft advisory message to impacted users, built from ticket fields (affected product and versions, vulnerability description, what users should do, and fixed versions once available) [1, Art. 14(8); Annex I, Part II(4)]

Excluded

- Submitting reports to ENISA or a CSIRT [1, Art. 14(7); Art. 16]. There is no public API available, so the submission will be a manual process.
- Severe incidents [1, Art. 14(3)-(4)] will use a separate final-report deadline which we will not be focusing on to stay within a one deadline model.
- System will not determine whether exploitation is real or when awareness occurred (both decided by human judgement and entered into the system).
- Each team may build its own intake (and any integrations related to that), but it is not part of the shared spec.
- Vulnerability detection, SBOM matching, deployment of fixes.

Limits

- One manufacturer per deployment
- One product per ticket
- At most 1,000 tickets per deployment
- English JSON input and output
- All times handled in UTC
- EU Member States only for where product is sold

Q2.3 Cost and feasibility estimate

Estimate the cost (number of person-hours) for each team to deliver this scope through Phase 6. Break the estimate into major work areas; include design, implementation, integration, validation, and deployment. State the available GenAI tools or licenses, expected use, and how they affect the estimate. Use ranges where uncertain; explain differences between teams.

Estimating 3-6 hours per person per week for the remaining phases. That works out to about 100-200 person-hours in total through Phase #6.

- Design: 15-25 hours
- Implementation: 35-75 hours
- Integration: 10-20 hours
- Validation: 30-60 hours
- Deployment: 10-20 hours

Team 17 has Claude Code (Quincy) and OpenAI Codex (Richard and Ann) Team 21 has Claude Code (Aashik) and OpenAI Codex (Aashik, Andrew, Ryan, Conrad). GenAI will help to build the project from the ground up, which should roughly save 40% on implantation time. Without it, we would estimate 1-2 more hours per person per week. All GenAI outputs will be validated and tested through both manual and automated pipelines.

Team 21 has four members, so the work can spread out more between individuals.

Q3 Common Specification

Shared work - complete with your pod.

Q3.1 Requirements

List every consequential requirement that each implementation must satisfy. Assign stable identifiers (REQ-01, REQ-02, and so on) and reuse them in later sections. Write each requirement as an observable behavior or a checkable constraint, including applicable conditions. Cite its regulatory source or explain the stakeholder need it addresses. Be careful about scope. If there are only a few requirements, you are likely missing important capability. If your list is too long, each requirement may be too narrowly written, or your spec may be too ambitious for a semester project. Cover the main successful behavior, invalid inputs and failures, important data or state, and applicable security, regulatory, and quality requirements. State measurable limits where needed. Identify categories that do not apply and explain why. Interface details may be defined once in Section 5 and referenced here. ***Wording:*** *Choose a pattern that fits. The examples below are optional. (Eg. For state-dependent behavior, use "While [state], the system shall [response].") See the EARS resource for other patterns: https://alistairmavin.com/ears/*

REQ-01 Intake: When a client submits a valid ticket, the system shall store it in state received, assign a unique ID, set the receipt time to the current system time, and return 201 with the ticket. Source - contact point for vulnerability reports [1, Art. 13(17); Annex I, Part II(6)].

REQ-02 Ticket Marked as Reportable: While ticket is in state received, when a person marks it as actively exploited with an awareness time and a reason, the system shall move it to a reportable state and start tracking deadlines from that awareness time (the awareness time may be earlier than the receipt time but not later than the current time). Source - [1, Art. 3(42); Art. 14(1)].

REQ-03 Ticket Marked as Not Reportable: While a ticket is in state received, whena person marks it as not actively exploited or invalid with a reason, the system shall close it, and no reporting deadlines should apply. Source – reporting applies only to actively exploited vulnerabilities [1, Art. 14(1)].

REQ-04 Deadline Computation: While a ticket is in a state reportable, the system shall set the early warning due time to the awareness time + 24 hours and the notification due time to the awareness time + 72 hours. Once a fix time is recorded, it shall set the final report due time to 14 days after the fix time, or 14 days after the awareness time if the fix came first. Times are exact elapsed hours in UTC (no adjustment for weekends, holidays, or daylight savings). Source - [1, Art. 14(2)(a)-(c)].

REQ-05 Obligation Status: For each of the three reports, the system shall show one status based on the current time: not_applicable (ticket not reportable), awaiting_fix (final report, no fix time yet), pending (not submitted, not yet due), overdue (not submitted, past due), submitted (on time), or submitted_late. Source - users need to see missed deadlines [1, Art. 14(2)].

REQ-06 Updates and Recalculation: While a ticket is not yet closed, the system shall accept updates to its details. Changing the awareness time shall require a reason and is allowed only while the ticket is reportable. Neither awareness time nor the fix time may be later than the current time. Any change to either shall recalculate the deadlines immediately. Source - scope often changes mid-incident [1, Art. 14(2)].

REQ-07 Report Drafts: For each repot type (early warning, notification, final report, and user advisory), the system shall return a draft built from the ticket's fields and list any required fields that are missing, as defined in the Q3.2 required-field table. Source - [1, Art.14(2)(a)-(c); Art. 14(8)].

REQ-08 Submission Recording: While a ticket is in a reportable state, when a person records that one of the three Article 14 reports was submitted, the system shall accept it only if no required fields are missing and the previous report (early warning > notification > final report) was already submitted, and shall then lock that report's content. Source - evidence of what was reported [1, Art. 14(1)-(2)].

REQ-09 Closure: While a ticket is in a reportable state, the system shall close it on request only after all three reports are recorded as submitted. While a ticket is closed, the system shall reject any changes but still allow it to be read. Source - [1, Art. 14(2)(c)].

REQ-10 Listing: The system shall list tickets, filterable by state and by report status, with the nearest unmet deadline first. Source - Users need to see what is due next, including reports that arrive outside of working hours.

REQ-11 Audit Log: When a request changes a ticket, the system shall add an entry to that ticket's log with the time, the person, the action, the old and new values, and any reason given. Entries shall never be edited or deleted except by a test reset (REQ-16). Source - [1, Annex I, Part II(1); Annex I, Part I(2)(l)].

REQ-12 Error Handling: When a request is invalid or not allowed in the ticket's current state, the system shall reject it with the status code and error format defined in Section 5, name the field at fault where applicable, and change nothing. Invalid input shall never cause a server error. Source - HTTP semantics [2] and problem details[3]; cross-team testing.

REQ-13 Time Format: The system shall accept RFC 3339 timestamps with any UTC offset and return all timestamps in UTC, to the second. Source - deadlines must match exactly across teams [4].

REQ-14 Authentication and HTTPS: The system shall accept requests only over HTTPS, reject any request without a valid access token (except for the health check), and require every change to name the person making it. Source – protection from unauthorized access [1, Annex I, Part I (2)(d)-(e)].

REQ-15 Reporter Confidentiality: The system shall show the reporter's name and contact details only when a single ticket is requested, and never in lists, drafts, advisories, logs, or error messages. Source - data confidentiality and minimization [1, Annex I, Part I(2)(e), (g)].

REQ-16 Test Controls: The system shall provide a reset operation that deletes all data and a clock operation that fixes the current time at a given value until it is changed. Source - repeatable cross-team testing.

REQ-17 Performance: With up to 1,000 tickets stored, 95% of requests shall be completed within 1 second, as measured by the test client. Source - usability during an incident; practical cross-team test runs.

REQ-18 Persistence: The system shall keep tickets, reports, and audit logs across a service restart. Source – compliance evidence must not be lost.

Categories that do not apply: privacy beyond reporter details (no other personal data is stored), availability guarantees beyond scheduled test windows (not a production service), scalability past 1,000 tickets, and accessibility or localization (no user interface in the shared spec).

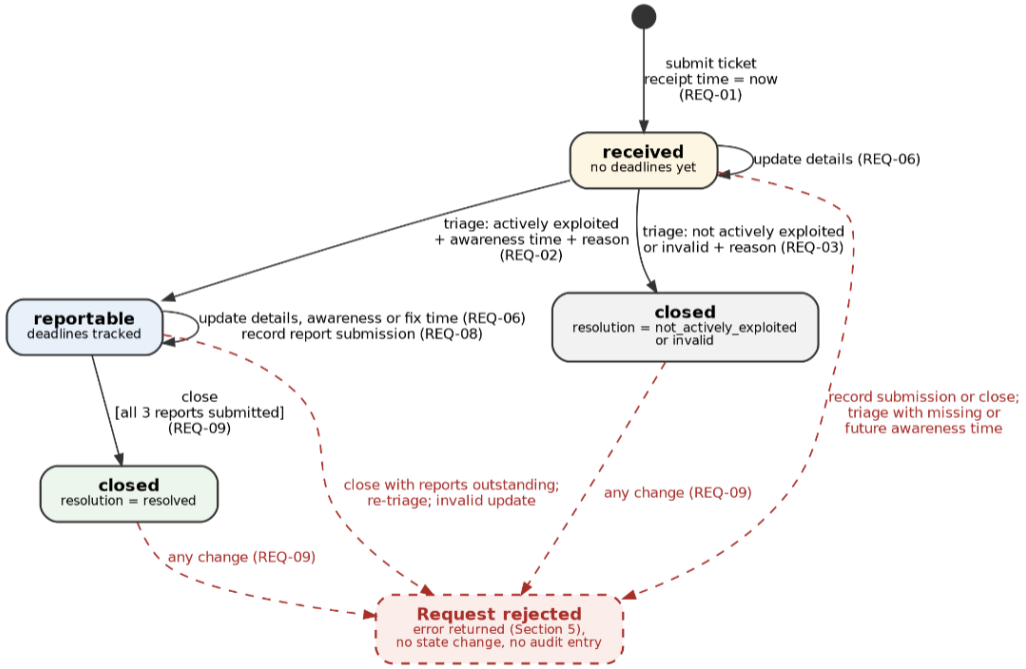
Q3.2 Behavioral and information models

Provide models precise enough to guide independent implementations. Show the main behavior, important data or state, and relevant failure paths. Suitable models include workflows, state machines, schemas, invariants, or structured examples. Use the views your system needs; do not reproduce the same information in multiple forms. Beside each model, list the requirement identifiers from Section 3.1 that it helps describe or clarify.

Model A — Ticket lifecycle

Clarifies: REQ-01, REQ-02, REQ-03, REQ-06, REQ-08, REQ-09, REQ-12

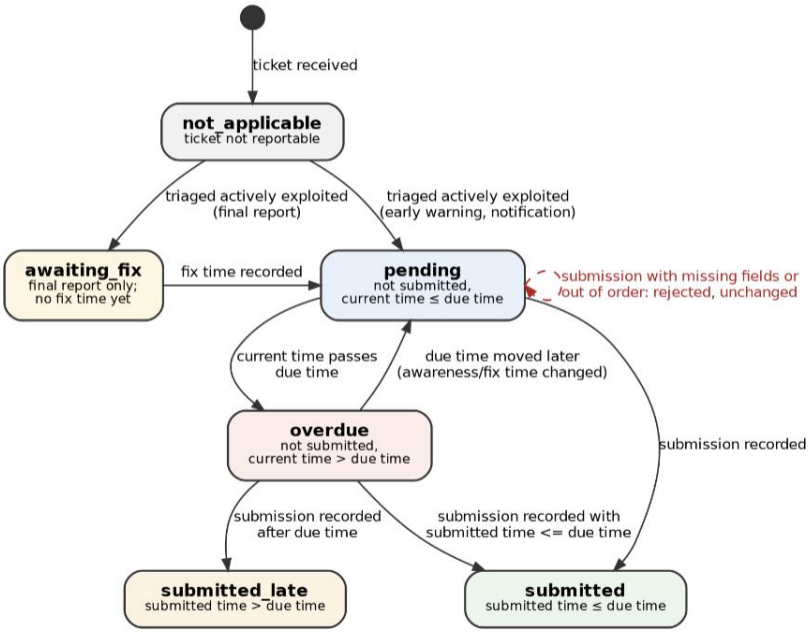
Each ticket is always in exactly one state. Black arrows are allowed requests; red dashed arrows are requests the system must reject without changing anything.



Model B — Report status (one instance per report: early warning, notification, final report)

Clarifies: REQ-04, REQ-05, REQ-06, REQ-08

Status is never stored. It is recalculated on every read from the stored awareness, fix, and submission times and the current system time, so arrows show how status can move, not events the system must detect. A ticket closed as not reportable keeps all three reports at not_applicable.



Model C — Required fields for each draft

Clarifies: REQ-07, REQ-08, REQ-15

R = required: listed in missing_fields when empty, and blocks recording a submission. **O** = included if present. **—** = not included. Every draft also contains the ticket ID and awareness time. The user advisory has no deadline and no submission; it is available once a ticket is reportable.

Ticket field	Early warning	Notification	Final report	User advisory	Basis in [1]
product.manufacturer, product.name	R	R	R	R	Art. 14(2)(b), 14(8)
product.affected_versions (at least one)	R	R	R	R	Art. 14(2)(b), 14(8)
member_states (at least one EU country)	R	R	R	—	Art. 14(2)(a)
vulnerability.cve_id	—	O	O	O	—
vulnerability.description	—	R	R	R	Art. 14(2)(b), (c)(i); 14(8)
exploit.description	—	R	R	—	Art. 14(2)(b)
mitigations.taken	—	R	R	—	Art. 14(2)(b)
mitigations.for_users	—	R	R	R	Art. 14(2)(b); 14(8)
sensitivity (TLP 2.0 label [5])	—	R	R	—	Art. 14(2)(b)
severity, impact	—	—	R	O	Art. 14(2)(c)(i)
malicious_actor	—	—	O	—	Art. 14(2)(c)(ii)
fix.available_at, fix.description	—	—	R	O	Art. 14(2)(c)(iii)
fix.fixed_versions (at least one)	—	—	R	O	Art. 14(2)(c)(iii)
reporter.name, reporter.contact	never	never	never	never	REQ-15

Model D — Deadline examples (shared expected outputs)

Clarifies: REQ-02, REQ-04, REQ-05, REQ-06, REQ-13

All times UTC. Deadlines are exact elapsed hours: 24 h, 72 h, and 14 days (336 h), with no adjustment for weekends, holidays, or daylight saving. Each team's implementation must return exactly these values.

#	Scenario	Input	Expected output
E1	Friday report, triaged Monday, awareness at receipt	Received 2026-10-02T17:00:00Z (Fri). Triaged at 2026-10-05T09:00:00Z with awareness 2026-10-02T17:00:00Z	Early warning due 2026-10-03T17:00:00Z (**overdue** at triage). Notification due 2026-10-05T17:00:00Z (**pending**)
E2	Same report, awareness at Monday confirmation	Same as E1, awareness 2026-10-05T09:00:00Z	Early warning due 2026-10-06T09:00:00Z. Notification due 2026-10-08T09:00:00Z
E3	Evidence of exploitation arrives after the report	Received 2026-10-05T09:00:00Z, awareness 2026-10-07T14:30:00Z	Early warning due 2026-10-08T14:30:00Z. Notification due 2026-10-10T14:30:00Z
E4	Fix after awareness	Awareness 2026-10-05T09:00:00Z, fix 2026-10-20T12:00:00Z	Final report due 2026-11-03T12:00:00Z (no daylight-saving shift)
E5	Fix released before awareness	Awareness 2026-10-05T09:00:00Z, fix 2026-09-28T00:00:00Z	Final report due 2026-10-19T09:00:00Z (14 days counted from awareness)
E6	Exact boundary	E2 early warning not submitted; clock at 2026-10-06T09:00:00Z, then 09:00:01Z	**pending** at the due time; **overdue** one second later
E7	Time zone offset in input	Awareness sent as 2026-10-05T11:00:00+02:00	Stored and returned as 2026-10-05T09:00:00Z
E8	Awareness time in the future (failure)	Clock 2026-10-05T09:00:00Z, awareness 2026-10-05T09:00:01Z	Rejected; ticket stays **received**; no deadlines set

E1 and E2 show the same ticket producing different deadlines depending on the awareness time people record at triage (see Q1.3).

Q3.3 Model explanation

***(4-5 sentences)**: Explain how your models fit together, and clarify any relationships or decisions that may not be so obvious when looking at your models. Additionally, explain any assumptions that, if left open, could cause teams to implement different behavior. (Record significant unresolved questions in Section 9.)*

Model A tracks the ticket as a whole, and Model B runs three times inside the reportable state, once for each Article 14 report; a ticket can only close once all three reports are submitted. Model C decides both what each draft contains and when a submission is blocked, and Model D gives the exact deadlines that Models A and B must produce. Report status is never stored but recalculated from stored times and the current time, so setting the test clock backward can move an overdue report back to pending. To keep teams from building different behavior, we fixed three assumptions: deadlines are exact UTC hours (no weekened or holiday adjustment), all three reports are always required even though Article 14(2)(b)-(c) allows skipping one when the information was already provided [1], and a fix released before awareness starts the 14-day clock at awareness (E5). We also require

every notification field even though the regulation says “as available”, so incomplete reports are flagged instead of silently skipped.

Q4 Acceptance Criteria

Shared work - complete with your pod.

Q4.1 Requirement coverage

For every requirement in Section 3, define evidence and a pass/fail rule. Assign acceptance-criterion identifiers (AC-01, AC-02, and so on) and list the requirement identifiers they evaluate. One criterion may cover multiple requirements, and one requirement may need multiple criteria. For each criterion, specify the setup or input, action or check, expected result, and method of evaluation. Include relevant failure or boundary cases. Quantify thresholds and tolerances when they affect acceptance. Identify which checks can run automatically and which require inspection, analysis, or human judgment.

Unless noted, every test first resets the service and fixes the clock (REQ-16). “Auto” means another team can run it automatically through the web interface.

AC-01 (REQ-01, REQ-13). Clock at 2026-10-02T17:00:00Z; submit two valid tickets. > Both return 201 in state received with receipt time 2026-10-02T17:00:00Z and different IDs. Auto.

AC-02 (REQ-01, REQ-12). Submit tickets with no title, an unknown source, a client-supplied receipt time, invalid JSON, and a body over 64 KB. > 400 naming the bad field (413 for the large body); ticket count unchanged. Auto.

AC-03 (REQ-02, REQ-04). Reproduce Model D examples E1–E5. > Every due time and status matches exactly, to the second. Auto.

AC-04 (REQ-02). Triage as actively exploited with no awareness time, a future awareness time (E8), and no reason; triage an already-reportable ticket. > 400, 422, 400, and 409; the ticket is unchanged each time. Auto.

AC-05 (REQ-03, REQ-09). Triage as not actively exploited, then try to change the ticket. > Ticket closed with all reports not_applicable; changes return 409; reading still works. Auto.

AC-06 (REQ-05). Step the clock through E6; record one report before its due time and one after. > pending at the due time, overdue one second later; submitted and submitted_late as appropriate; final report awaiting_fix until a fix time is set. Auto.

AC-07 (REQ-06). Change the awareness time with and without a reason; set a future fix time; change the awareness time on a received ticket. > Deadlines recalculated with an audit entry; 422; 422; 409. Auto.

AC-08 (REQ-07, REQ-15). Fill ticket fields one at a time and read all four drafts after each step. > Missing fields match Model C exactly at every step; no reporter details in any draft. Auto.

AC-09 (REQ-08). Record the notification before the early warning; the early warning with no member states; a valid early warning; the same one again; the final report with no fix time; then edit the product name. > 409, 422, 200, 409, 409; the recorded early warning's content is unchanged. Auto.

AC-10 (REQ-09). Close with the final report outstanding; then record all three reports and close. > 409, then closed with resolution resolved. Auto.

AC-11 (REQ-10). Create four tickets: one received, one overdue, two pending with different due times. List with each filter. > Exactly the expected tickets, nearest deadline first. Auto.

AC-12 (REQ-11). Run a scripted lifecycle with a known number of successful and failed requests; try to edit or delete the log. > One entry per successful change, numbered 1 to n with no gaps, with matching person and time; failed requests add nothing; edit or delete returns 405. Auto.

AC-13 (REQ-12). Send at least 500 malformed requests generated from the OpenAPI file. > No 5xx responses; every error uses the Section 5 format; no data changes. Auto.

AC-14 (REQ-13). Reproduce E7; send 2026-10-05 09:00 and 2026-10-05T09:00:00.789Z. > E7 value returned; 400; stored as 2026-10-05T09:00:00Z. Auto.

AC-15 (REQ-14). Call every endpoint with no token and a wrong token; make a change without naming the person; send a request over plain HTTP. > 401 everywhere except the health check; 400; plain HTTP refused or redirected with no data returned. Auto.

AC-16 (REQ-15). Set the reporter's contact to a unique string, call every read endpoint, and trigger errors on that ticket. > The string appears only in the single-ticket response. Auto.

AC-17 (REQ-16). Create tickets, reset, and list; set the clock and read it twice two seconds apart; submit a ticket. > Empty list; both reads identical; receipt time equals the set clock. Auto.

AC-18 (REQ-17). Load 1,000 tickets and send 200 mixed requests one at a time. > 95th-percentile response time ≤ 1 second. Auto.

AC-19 (REQ-18). Create a ticket with a recorded report, restart the service, and read it. > Ticket, report, and audit log unchanged. Demonstration by the owning team (recorded), since other teams can't restart it.

Q4.2 Test data and expected outputs

***(3-4 sentences)**: Explain where representative valid, invalid, and boundary-case data will come from and how the pod will establish trustworthy expected outputs. Explain how the evaluation will remain repeatable if external services, changing datasets, or nondeterministic AI behavior are involved.*

All test data will be made up: valid cases come from the Model D deadline examples, and the Model C field table, and invalid boundary cases come from the limits in the Section 5 interface (missing fields, wrong types, oversized requests, exact deadline boundaries). The Pod keeps these in one shared test file, and each expected output is calculated by hand by one team and checked by the other before it's added. If an implementation disagrees with a test, we first check whether the spec is wrong and fix it there. Tests stay repeatable because each one resets the service and fixes the clock, the system calls no external services, and no AI-generated content is part of any tested output.

Q4.3 Independent evaluation

***(2-3 sentences)**: Explain which criteria another team can evaluate through the common web service interface (which we will define in section 5) without knowing your internal design. For criteria requiring other forms of evidence, identify the artifact and how it will be checked. A credible evaluation plan is required here; completed tests and results are not yet required.*

AC-01 through AC-18 can all be run by another team through the common web interface, using only the base URL, an access token, and the shared test file, with no knowledge of how the service is built. AC-19 (persistence) needs a service restart that only the owning team can perform, so that team will submit a recorded demonstration of creating a ticket, restarting the service, and reading the ticket back unchanged. The Pod will review the recording against the expected result in AC-19.

Q5 Cross-Team Testing Interface

Shared work - complete with your pod.

Define the common web-service interface that every team will provide. Automated tests must be able to send requests to any team's deployed service and check the observable results. A network-accessible implementation is required beginning with Phase 3.

Q5.1 Interface contract

Provide an OpenAPI document or an equivalent structured HTTP specification covering every operation needed for cross-team testing. For each operation, state its purpose and related requirement identifiers, HTTP method and path, request format, successful response, and error responses with status codes. Define relevant state changes and behavior needed to interpret results.

Every request except the health check requires a bearer token. Every request that changes data requires an X-Actor header naming the person making the change.

1. Health check

   GET /health

   Request: none

   Success: 200

   Errors: none

   Requirements: none

2. Submit ticket

   POST /tickets

   Request: ticket fields (source, title, description, product; others optional)

   Success: 201 with the new ticket

   Errors: 400, 401, 413

Requirements: REQ-01, REQ-12, REQ-13, REQ-14

3. List tickets

GET /tickets?state=...&obligation_status=...

Request: optional filters

Success: 200 with a list of tickets, nearest deadline first

Errors: 400, 401

Requirements: REQ-10, REQ-15

4. Get ticket

GET /tickets/{id}

Request: none

Success: 200 with the ticket, deadlines, and report statuses

Errors: 401, 404

Requirements: REQ-04, REQ-05

5. Update ticket

PATCH /tickets/{id}

Request: changed fields; a reason is required if the awareness time changes

Success: 200 with the updated ticket

Errors: 400, 401, 404, 409, 413, 422

Requirements: REQ-04, REQ-06

6. Triage ticket

POST /tickets/{id}/triage

Request: decision, reason, and awareness time (required if actively exploited)

Success: 200 with the ticket in state reportable or closed

Errors: 400, 401, 404, 409, 422

Requirements: REQ-02, REQ-03

7. Get draft

GET /tickets/{id}/reports/{type}

Request: type is early_warning, notification, final_report, or user_advisory

Success: 200 with the draft and a list of missing required fields

Errors: 401, 404, 409

Requirements: REQ-07, REQ-15

8. Record submission

POST /tickets/{id}/reports/{type}/submission

Request: type is early_warning, notification, or final_report; optional submitted time and reference number

Success: 200 with the locked report

Errors: 400, 401, 404, 409, 422

Requirements: REQ-05, REQ-08

9. Close ticket

POST /tickets/{id}/close

Request: none

Success: 200 with the ticket in state closed

Errors: 400, 401, 404, 409

Requirements: REQ-09

10. Get audit log

GET /tickets/{id}/audit

Request: none

Success: 200 with the list of audit entries

Errors: 401, 404, 405

Requirements: REQ-11

11. Reset (testing)

POST /test/reset

Request: none

Success: 204

Errors: 401

Requirements: REQ-16

12. Clock (testing)

GET /test/clock, PUT /test/clock, DELETE /test/clock

Request: PUT takes the time to fix the clock at

Success: 200, 200, 204

Errors: 400, 401

Requirements: REQ-16

Error codes:

400 = request does not match the schema

401 = missing or invalid token

404 = unknown ticket or draft type

405 = method not allowed

409 = not allowed in the ticket's current state

413 = body over 64 KB

422 = valid request that breaks a rule (for example, an awareness time in the future)

If several apply, the first in the order 401, 413, 404, 400, 409, 422 is returned.

State changes: Submit, triage, update, record submission, and close each change data and add one audit entry when they succeed. Reads never change data, and a failed request changes nothing. Reset deletes all data, and the clock operations fix or release the current time.

Q5.2 Data schemas and examples

Define the fields, types, required/optional status, and allowed values or constraints for the inputs and outputs. Specify units and formats where relevant. Include at least one valid request/response example for each core operation and one invalid-input example with its expected error. Cross-reference definitions in the contract rather than duplicating them.

All request and response bodies are JSON, up to 64 KB. Timestamps use RFC 3339 [4] with any UTC offset and are always returned in UTC to the second (for example, 2026-10-05T09:00:00Z). Errors use the problem details format [3]. The fields below are the ones tests depend on.

TICKET FIELDS (sent by the client)

- source (required): one of external_researcher, customer, internal, scanner, csirt

- title (required): text, 1-200 characters

- description (required): text, 1-10,000 characters

- product.manufacturer, product.name (required): text, 1-200 characters

- product.affected_versions (optional): list of up to 50 version strings, each 1-100 characters

- reporter.name, reporter.contact (optional): text, up to 200 characters; returned only by Get ticket

- member_states (optional): list of unique EU country codes (ISO 3166-1 alpha-2, e.g. DE, FR, NL)

- vulnerability.cve_id (optional): format CVE-YYYY-NNNN (4 or more digits at the end)

- vulnerability.description, exploit.description, mitigations.taken, mitigations.for_users, impact, malicious_actor (optional): text, 1-10,000 characters

- sensitivity (optional): CLEAR, GREEN, AMBER, AMBER+STRICT, or RED (TLP 2.0 [5])

- severity (optional): low, medium, high, or critical

- fix.available_at (optional): timestamp, not later than now

- fix.description (optional): text, 1-10,000 characters

- fix.fixed_versions (optional): list of up to 50 version strings

TICKET FIELDS (set by the system; sending them returns 400)

- id: text; tests must not assume a format

- state: received, reportable, or closed

- resolution: resolved, not_actively_exploited, invalid, or null

- received_at: timestamp set when the ticket is submitted

- aware_at: timestamp set by triage, or null

- obligations.early_warning / .notification / .final_report: each has status, due_at, submitted_at, and reference

- status values: not_applicable, awaiting_fix, pending, overdue, submitted, submitted_late (see Q3.2 Model B)

- next_due_at: earliest unmet due time, or null

OTHER REQUESTS

- Triage: decision (required: actively_exploited, not_actively_exploited, or invalid), reason (required, 1-10,000 characters), aware_at (timestamp, required if actively_exploited, not later than now)

- Update: any client ticket field except source, plus change_reason (required when aware_at changes). Nested objects are merged one level deep, lists are replaced whole, and null clears an optional field.

- Record submission: submitted_at (optional timestamp, defaults to now, between aware_at and now), reference (optional, 1-200 characters)

- Set clock: now (required timestamp)

OTHER RESPONSES

- Draft: ticket_id, type, status, due_at, submitted_at, reference (the last four are null for user_advisory), content (the fields listed in Q3.2 Model C, never reporter), missing_fields (list of empty required field names)

- Audit entry: seq (integer starting at 1), at (timestamp), actor, action (created, triaged, updated, report_submitted, or closed), changes (list of field, old value, new value), reason (or null)

- Error: title, status, optional detail, and for 400 and 422 an errors list of {field, message}

EXAMPLES

(The Authorization header is left out for cleanlines. Every request except /health needs it.)

Example 1 - Submit ticket (clock at 2026-10-05T09:00:00Z)

POST /tickets

X-Actor: intake-form

{

  "source": "external_researcher",

  "title": "Auth bypass in admin API",

  "description": "Researcher reports login bypass using a crafted session cookie.",

  "product": { "manufacturer": "Acme", "name": "Acme Router OS", "affected_versions": ["4.1.0", "4.1.1"] },

  "reporter": { "name": "J. Doe", "contact": "jdoe@example.org" }

}

Response: 201

{

  "id": "t_8f2a", "state": "received", "resolution": null,

  "received_at": "2026-10-05T09:00:00Z", "aware_at": null,

  "obligations": {

    "early_warning": { "status": "not_applicable", "due_at": null, "submitted_at": null, "reference": null },

    "notification": { "status": "not_applicable", "due_at": null, "submitted_at": null, "reference": null },

"final_report": { "status": "not_applicable", "due_at": null, "submitted_at": null, "reference": null }

},

"next_due_at": null,

... (submitted fields echoed back; all other fields null or empty)

}

Example 2 - Triage (clock at 2026-10-07T15:00:00Z; matches Model D example E3)

POST /tickets/t_8f2a/triage

X-Actor: q.dekl

{ "decision": "actively_exploited", "aware_at": "2026-10-07T14:30:00Z", "reason": "Customer logs show exploitation." }

Response: 200 with state "reportable", aware_at "2026-10-07T14:30:00Z",

early_warning { "status": "pending", "due_at": "2026-10-08T14:30:00Z" },

notification { "status": "pending", "due_at": "2026-10-10T14:30:00Z" },

final_report { "status": "awaiting_fix", "due_at": null },

next_due_at "2026-10-08T14:30:00Z"

Example 3 - Get draft

GET /tickets/t_8f2a/reports/early_warning

Response: 200

{

"ticket_id": "t_8f2a", "type": "early_warning", "status": "pending",

"due_at": "2026-10-08T14:30:00Z", "submitted_at": null, "reference": null,

```
  "content": {
    "ticket_id": "t_8f2a", "aware_at": "2026-10-07T14:30:00Z",
    "product": { "manufacturer": "Acme", "name": "Acme Router OS", "affected_versions":
["4.1.0", "4.1.1"] },
    "member_states": []
  },
  "missing_fields": ["member_states"]
}
```


Example 4 - Update ticket

PATCH /tickets/t_8f2a

X-Actor: q.dekl


```
{ "member_states": ["DE", "NL"] }
```


Response: 200 with member_states ["DE", "NL"]; the early warning draft now has missing_fields []

Example 5 - Record submission (clock at 2026-10-08T10:30:00Z)

POST /tickets/t_8f2a/reports/early_warning/submission

X-Actor: q.dekl


```
{ "submitted_at": "2026-10-08T10:00:00Z", "reference": "SRP-2026-00417" }
```


Response: 200 with status "submitted", submitted_at "2026-10-08T10:00:00Z", and the content locked

Example 6 - Close ticket (after all three reports are recorded)

POST /tickets/t_8f2a/close

X-Actor: q.dekl

Response: 200 with state "closed" and resolution "resolved"

INVALID EXAMPLES

Example 7 - Schema violation (400)

POST /tickets

X-Actor: intake-form

{ "source": "email", "description": "x", "product": { "manufacturer": "Acme", "name": "Router" } }

Response: 400

{

  "title": "Invalid request", "status": 400,

  "errors": [

    { "field": "title", "message": "required" },

    { "field": "source", "message": "must be one of external_researcher, customer, internal, scanner, csirt" }

  ]

}

Example 8 - Rule violation (422) and state conflict (409)

Recording the early warning before member_states is set returns 422 with errors[0].field = "member_states".

Recording the notification before the early warning returns 409.

In both cases the ticket is unchanged and no audit entry is added.

Q5.3 Repeatable test setup

***(3-5 sentences)**: Explain how tests initialize or reset data, obtain any required authentication, and run against a selected team's base URL. Identify any shared fixtures, external-service assumptions, or ordering constraints. If the service is stateless or needs no authentication/reset, explicitly say so. Keep internal architectures and implementation technologies independent.*

Each team publishes its deployed base URL in a shared pod file and sends the other team an access token privately (never committed to a repository), so a test run needs only a base URL and a token. Every test starts by calling POST /test/reset and then PUT /test/clock to fix the current time, so tests do not depend on each other and can run in any order. Because reset clears the whole service, only one test suite may run against a deployment at a time. The only shared fixture is the Pod's test file of inputs and expected outputs (Q4.2), and no test depends on any external service. The test endpoints must stay enabled on the deployment used for evaluation, and nothing in this setup constrains a team's language, framework, database, or hosting.

Q6 Milestones for Phases 3-5

Shared work - complete with your pod.

For each phase, identify the capability to deliver, artifacts to produce or update, uncertainty to reduce, demonstration of completion, and automated validation to add. Use requirement and acceptance-criterion identifiers. Each milestone should produce a deployable, testable increment. The instructors will review the plan and may require scope or sequencing changes.

Q6.1 Phase 3

Capability: A deployed HTTPS service with authentication, test controls, ticket submission, triage, and deadline calculation (REQ-01 to REQ-04, REQ-12 to REQ-14, REQ-16).

Artifacts: Deployed base URL in the shared pod file; shared OpenAPI file frozen for these operations; shared test file with the Model D examples; team CI pipeline.

Uncertainty reduced: Whether both teams can deploy a reachable service, and whether the spec is precise enough that two independent implementations produce identical deadlines.

Demonstration: The pod runs one shared test suite against both deployments, and both pass.

Automated validation: AC-01 to AC-05, AC-14, AC-15, AC-17, plus automatic checks that responses match the OpenAPI schemas.

Q6.2 Phase 4

Capability: The full reporting workflow: updates, report status, all four drafts, submission recording, closure, listing, audit log, and reporter confidentiality (REQ-05 to REQ-11, REQ-15).

Artifacts: Updated OpenAPI file if changes are needed; each team's cross-team test suite for its podmate; updated shared test file.

Uncertainty reduced: Whether both teams interpret the required-field rules, submission order, and state edge cases the same way.

Demonstration: A scripted incident (submit, triage, three submissions, close) passes against the podmate's deployment.

Automated validation: AC-06 to AC-13, AC-16; first full cross-team run, with any disagreements resolved in the spec.

Q6.3 Phase 5

Capability: Performance, persistence, each team's own intake channel, and a stable final version (REQ-17, REQ-18).

Artifacts: Performance results; recorded restart demonstration; final spec version; full cross-team test results; short operations guide.

Uncertainty reduced: Whether hosting meets the performance target, whether data survives restarts, and whether both implementations still pass every test after late changes.

Demonstration: Performance run meets the 1-second target, the restart recording shows data unchanged, and the full suite passes against both deployments.

Automated validation: AC-18, plus AC-19 by recorded demonstration; the full suite (AC-01 to AC-18) runs in CI on every change and against the podmate before each deadline.

Q6.4 Sequencing rationale

***(4-5 sentences)**: Explain why this order is feasible and how early milestones reduce risks or establish dependencies for later work. Identify any scope reduction the pod would make if progress falls behind.*

Phase 3 tackles the two things most likely to break cross-team testing, deployment and deadline calculation, because every later test depends on a reachable service and on both teams computing identical deadlines. Phase 4 builds the rest of the reporting workflow on that base, and its first full cross-team run is where differences in how each team reads the spec will show up, early enough to fix the spec before Phase 5. Phase 5 holds work that nothing else depends on, such as performance tuning, restart testing, and each team's

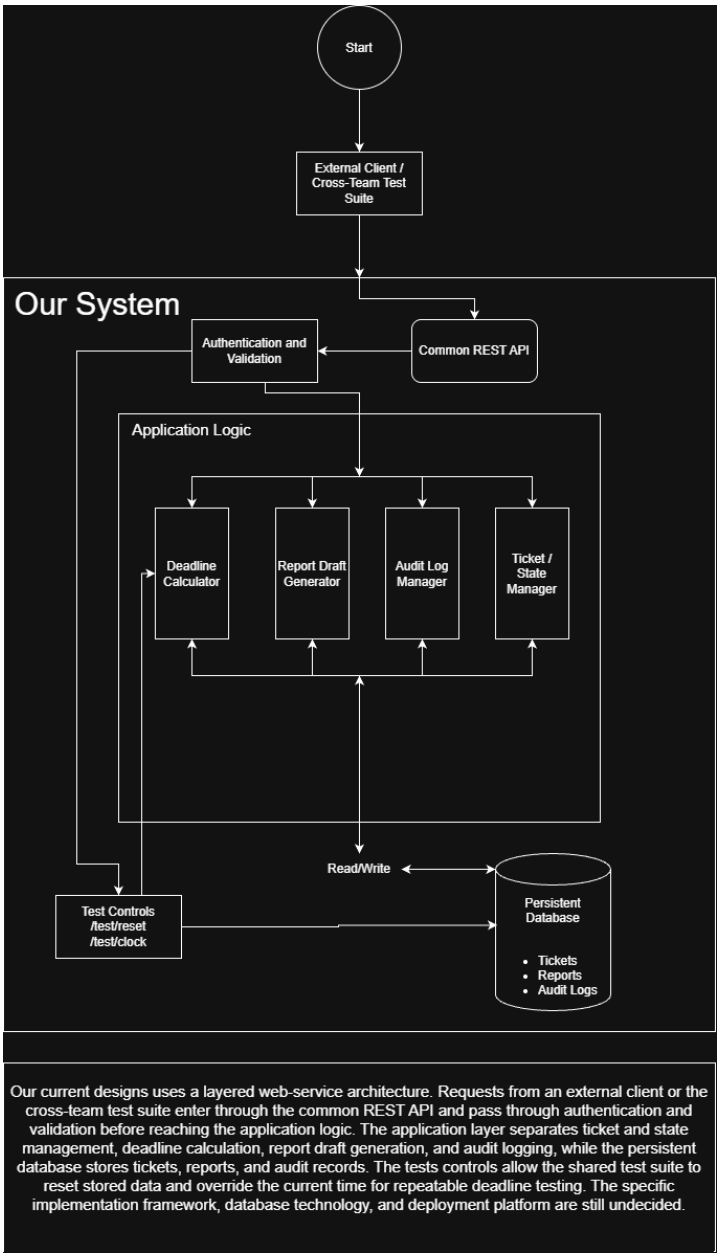
intake channel, which leaves room to absorb problems found in Phase 4. If progress falls behind, the pod would cut in this order: limit each team's intake to calling the API directly, replace the performance target (REQ-17) with a measured baseline, and drop the filtering and ordering in listing (REQ-10). The core Article 14 behavior (REQ-01 to REQ-09, REQ-11) will not be cut.

Q7 Team System Design

Individual team work - complete Sections 7-9 within your team.

Q7.1 Architecture and data flow

Provide a rough sketch showing your team's current idea of how the system could work. Label the main parts, show how information might move between them, and indicate where requests through the pod's common web-service interface would enter the system. Add a brief explanation of the approach and identify anything still undecided. A clearly labeled diagram is sufficient.



Q7.2 Design decisions

***(two decisions, 3-4 sentences each)**: For each consequential design decision, state your choice, an alternative considered, and the tradeoff that led to your choice. Reference the requirement or quality attribute affected. Your initial design may evolve as you learn.*

1. We are planning to implement this system as a single deployed web service with separate internal modules for ticket/state management, deadline calculation, report generation, and audit logging. An alternative would be to separate these responsibilities into multiple independently deployed microservices. We decided to go with a single-service approach because it reduces deployment and communication complexity while still keeping the code separated by responsibility, which is more appropriate for the size of this semester's project. This decision supports reliablity and performance requirements such as REQ-17 while making cross-team testing through the common API easier.

The plan is to use a persistent relational database to store tickets, report submissions, and audit records. Alternatives we considered include keeping data only in memory or storing records directly in JSON files, which would be simpler initially but could make persistence, querying, and consistent updates more difficult. A relational database provides structured storage and makes it easier to preserve relationships between tickets, reports, and audit entries while ensuring that data survives applications restarts. This decision directly supports REQ-10 for ticket filtering and ordering, REQ-11 for audit logging, and REQ-18 for persistence.Q7.3 Operation and observability

***(3-5 sentences)**: Explain how your team plans to deploy and run the web service, manage persistent data and required secrets/access, and detect failures. Identify the logs, metrics, or other signals needed to diagnose incorrect behavior. If the system stores no persistent data, state that.*

We plan to deploy the service as a single HTTPS web service with a persistent relational database so that tickets, reports, and audit logs remain available across restarts. Access to the service will be protected using bearer tokens, and secrets such as access tokens or database credentials will be stored outside the source code using environmental variables or the deployment platform's secret management features. The application will also record logs for incoming requests, validation failures, authentication failures, state changes, and unexpected server errors so incorrect behavior can be traced. We will also monitor basic signals such as response status codes, request latency, and service availability to detect failures and confirm that our system continues to meet the required performance and reliability expectations. We will run a separate development deployment for our own testing and keep the evaluation deployment for Pod and scored tests runs, and application logs will record ticket IDs rather than request bodies so reporter contact details never appear in them (REQ-15).

Q7.4 Validation strategy

***(3-4 sentences)**: Describe a small prototype, experiment, or test your team could try first to validate its components and integrated system against Section 4. Identify the static checks, dynamic tests, or other analyses appropriate to the design and when they will run. Explain how failures will be investigated and regressions prevented.*

We will prototype the core ticket lifecycle by implementing ticket creation, triage, and deadline calculation, then compare the outputs against the shared Phase 2 test cases and acceptance criteria. Static checks like schema validation and code-quality checks will be run before integration while automated dynamic tests will exercise API endpoints, invalid inputs, state transitions, and deadline boundary cases. These tests will run automatically whenever code changes are pushed so that failures are detected early. When a failure does occur, we will compare the observed output with the shared specification and add a regression test before fixing the implementation to prevent the same issue from returning.

Q8 Build Buy and Reuse

Individual team work - complete within your team.

Identify at least one major capability (a capability or dependency such that its failure or replacement would affect the outcome of a requirement, security, cross-team testing, delivery effort, or schedule), and explain how your team will obtain each of them. Consider building it, using a library/framework, integrating an existing system, using a hosted service, or delegating suitable work to a GenAI service. Refer to Phase 1 research where it applies.

Q8.1 Capability decisions

For each major capability or dependency, identify the selected approach and candidate tool/service, explain why it fits, and describe the integration work your team must do. Address the cost, licensing, security, reliability, and maintenance implications that matter to the decision. Cite technical documentation or other supporting evidence.

REST API framework: We are planning to use FastAPI to implement a REST API instead of building an HTTP request handling, request validation, and API documentation ourselves. FastAPI supports OpenAPI and JSON Schema and provides us with automatic interactive API documentation which fits the pod's requirement for a structured and testable web-service interface [6]. Integration will require us to define the endpoints and request/response models from Section 5 and connect them to our ticket-management logic. FastAPI is distributed under the MIT license, so it does not add a licensing cost, but our team will still be responsible for maintaining dependencies and correctly configuring authentication and HTTPS [6].

We are planning to use SQLite with SQLAlchemy for storing tickets, reports, and audit records instead of creating our own file storage system. SQLite is a self-contained, serverless, zero-configuration relational database that normally stores the complete database in a single disk file which should be sufficient for the scope of this project's small deployment size and reduce database administration work [7]. SQLAlchemy will provide and object relational mapping layer so our Python objects persist and query through relational tables without us having to write all database operations manually [8]. SQLite is a public domain and does not require licensing fees but the deployment will need persistent disk storage so that the database survives application restarts [7].

Q8.2 GenAI use

***(3-4 sentences)**: State how your team expects to use GenAI during engineering and whether AI is part of the running system. Identify available tools/licenses, work delegated, and how outputs will be checked. Describe any effect on data handling, cost, or reliability. If you do not plan to use GenAI for a category, state that.*

Our team will be using GenAI tools such as ChatGPT and Claude Code (both through existing subscriptions purchased for this class) during specification and development for tasks including generating initial code scaffolding, suggesting test cases, debugging errors, reviewing code, and helping produce documentation. However GenAI will not be part of the deployed system and definitely not used to make vulnerability-reporting or compliance decisions, so the application will behave deterministically during cross-team testing. All AI-generated code and suggestions will be reviewed by a team member and validated during the shared acceptance tests before being incorporated into the project. WE will use only simulated project data when working with external AI services which avoids sending real vulnerability reports, credentials, or other sensitive information to those services.

Q9 Major Risks and Investigation Plan

Individual team work - complete within your team.

Identify the uncertainties most likely to prevent successful delivery. Consider technical feasibility, regulatory ambiguity, integration, security/privacy, AI reliability, performance, test data, validation, and external dependencies.

Q9.1 Priority risks

Describe two major risks, ordered by priority. For each, state the uncertainty , the effect if your team's assumptions are wrong, and a concrete investigation proposal. Identify who will investigate, when it must be resolved, and the evidence that would support a decision. Link to a milestone in Section 6 and include a fallback if the risk cannot be reduced

There are two major risks to our proposal. For one, our interpretation with the relationship between awareness time, fix availability, the 14-day final report deadline, and the information needed in that report may not suit the CRA's requirements. If said assumptions are wrong, both teams could implement the wrong interpretation, producing a product that does not meet the CRA's requirements. During Phase 3, Team 17 should review relevant CRA provisions and come back to look at Model C and D to ensure that it follows the given requirements. A report should be made dedicated to making sure the interpretation is not ambiguous enough for error to occur. If the risk cannot be reduced, then we should list our assumptions made during the process of coming up with our solution.

A second major risk is whether cross-team implementation will be compatible or not. If our programs cannot work with each other, it would jeopardize the project even if the logic should theoretically work. Team 17 will build its Phase 3 prototype, and compatibility will be tested at the end of Phase 3, when the pod runs the shared test suite against both deployments (Q6.1), then fully during the Phase 4 cross-team run (Q6.2). If both deployments pass the shared test suite, that is evidence that implementations are compatible. If there are compatibility issues, then the teams should head back to discuss what they can do to fix the problem as soon as possible.

Q9.2 Risks requiring pod agreement

(2-3 sentences)*: From Section 9.1, identify the risks that could require a change to the shared scope, requirements, acceptance criteria, or interface. Explain what the pod needs*

to decide and when. If none is currently identified, state that. Keep the common specification consistent when decisions change.

If the first major risk were to become a serious issue then that would require a change to the shared scope and requirements. We would need to seriously rework many of our components should the project not fulfill the demanded requirements the project asks for. By Phase 4, if the project implementation does not correlate with demands, then the pod will need to decide what next to do, and whether or not they want to rework the project.

Q10 References

Include 5–8 distinct, relevant sources. Number sources in the order they first appear in your report, and reuse the same number whenever you cite that source again. Place citations immediately after the supported claim and identify a section or provision when needed: Regulatory claim: "The obligation includes documenting product components [1, Annex I, Part II(1)]." Interface description: "OpenAPI provides a structured description of an HTTP API [1]." Protocol behavior: "A 400 response indicates a perceived client error [2, Section 15.5.1]."

Format each reference as follows: [Number] Author or issuing organization. "Title." Publication date or version, if available. URL. Accessed: YYYY-MM-DD.

Example: [1] European Parliament and Council of the European Union. "Regulation (EU) 2024/2847 (Cyber Resilience Act)." October 23, 2024. https://eur-lex.europa.eu/eli/reg/2024/2847/oj. Accessed: 2026-09-08.

[1] European Parliament and Council of the European Union. "Regulation (EU) 2024/2847 (Cyber Resilience Act)." October 23, 2024. https://eur-lex.europa.eu/eli/reg/2024/2847/oj. Accessed: 2026-09-27.

[2] Internet Engineering Task Force (IETF). "RFC 9110: HTTP Semantics." June 2022. https://www.rfc-editor.org/rfc/rfc9110. Accessed: 2026-09-27.

[3] Internet Engineering Task Force (IETF). "RFC 9457: Problem Details for HTTP APIs." July 2023. https://www.rfc-editor.org/rfc/rfc9457. Accessed: 2026-09-27.

[4] Internet Engineering Task Force (IETF). "RFC 3339: Date and Time on the Internet: Timestamps." July 2002. https://www.rfc-editor.org/rfc/rfc3339. Accessed: 2026-09-27.

[5] Forum of Incident Response and Security Teams (FIRST). "Traffic Light Protocol (TLP) Version 2.0." August 2022. https://www.first.org/tlp/. Accessed: 2026-09-27.

[6] FastAPI. "FastAPI Documentation." https://fastapi.tiangolo.com/. Accessed: 2026-09-27.

[7] SQLite. "About SQLite." https://sqlite.org/about.html. Accessed: 2026-09-27.

[8] SQLAlchemy. "SQLAlchemy ORM." Version 2.1.1. https://docs.sqlalchemy.org/en/21/orm/. Accessed: 2026-09-27.